Deep Learning - Lab 1: Regression and Classification with PyTorch

Name: Azzaoui Mohamed Omar
Program: Master Security IT & Big Data
Course: Deep Learning
Professor: Pr. ELAACHAK Lotfi

For this lab we worked locally in a Jupyter notebook, on CPU, with the data stored in a local folder pushed to github.

Objective

The goal of this lab is to get familiar with the PyTorch library by building DNN/MLP architectures for regression and classification tasks.

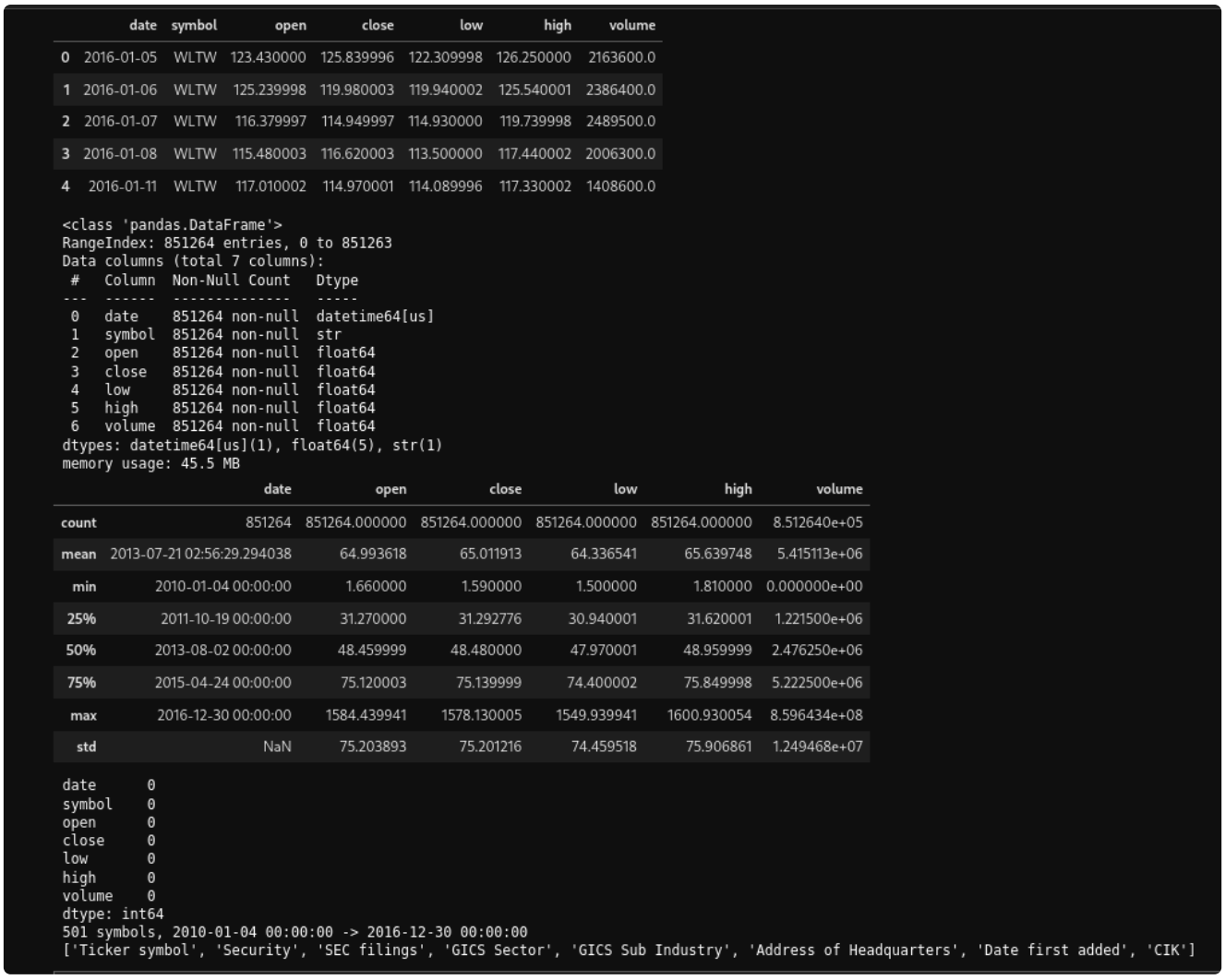
Setup

We installed all the libraries in one local environment, listed in `requirements.txt`. The NYSE dataset from Kaggle was downloaded to the folder `../NewYorkStock`, so the data must be placed there to run the notebook.

Part 1 - Regression

1. Exploratory Data Analysis

We used the file `prices-split-adjusted.csv`. It has 851,264 rows and 7 columns: `date`, `symbol` and five numeric columns (`open`, `close`, `low`, `high`, `volume`). It covers 501 companies from 2010-01-04 to 2016-12-30, and there are no missing values, so no imputation was needed. We also found no duplicated (date, company) rows, and no row where the high is below the open or close, or the low above them.



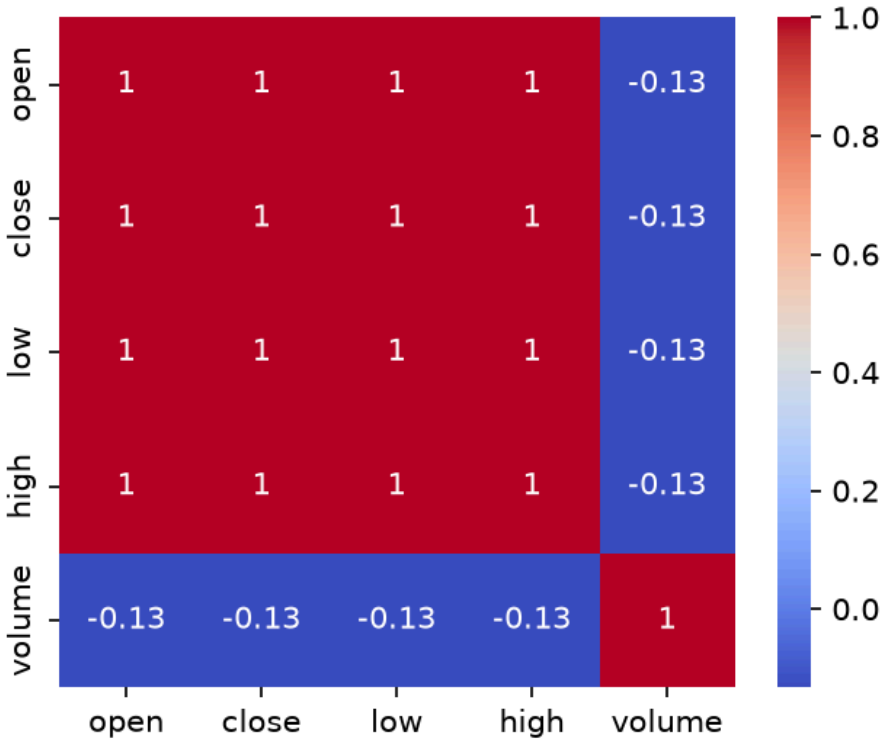
```
         date symbol      open      close       low      high    volume
0  2016-01-05   WLTW 123.430000 125.839996 122.309998 126.250000 2163600.0
1  2016-01-06   WLTW 125.239998 119.980003 119.940002 125.540001 2386400.0
2  2016-01-07   WLTW 116.379997 114.949997 114.930000 119.739998 2489500.0
3  2016-01-08   WLTW 115.480003 116.620003 113.500000 117.440002 2006300.0
4  2016-01-11   WLTW 117.010002 114.970001 114.089996 117.330002 1408600.0

<class 'pandas.DataFrame'>
RangeIndex: 851264 entries, 0 to 851263
Data columns (total 7 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   date    851264 non-null  datetime64[us]
 1   symbol  851264 non-null  str
 2   open    851264 non-null  float64
 3   close   851264 non-null  float64
 4   low     851264 non-null  float64
 5   high    851264 non-null  float64
 6   volume  851264 non-null  float64
dtypes: datetime64[us](1), float64(5), str(1)
memory usage: 45.5 MB
                                date           open          close            low           high        volume
count                         851264  851264.000000  851264.000000  851264.000000  851264.000000  8.512640e+05
mean   2013-07-21 02:56:29.294038      64.993618      65.011913      64.336541      65.639748  5.415113e+06
min          2010-01-04 00:00:00       1.660000       1.590000       1.500000       1.810000  0.000000e+00
25%          2011-10-19 00:00:00      31.270000      31.292776      30.940001      31.620001  1.221500e+06
50%          2013-08-02 00:00:00      48.459999      48.480000      47.970001      48.959999  2.476250e+06
75%          2015-04-24 00:00:00      75.120003      75.139999      74.400002      75.849998  5.222500e+06
max          2016-12-30 00:00:00    1584.439941    1578.130005    1549.939941    1600.930054  8.596434e+08
std                          NaN      75.203893      75.201216      74.459518      75.906861  1.249468e+07

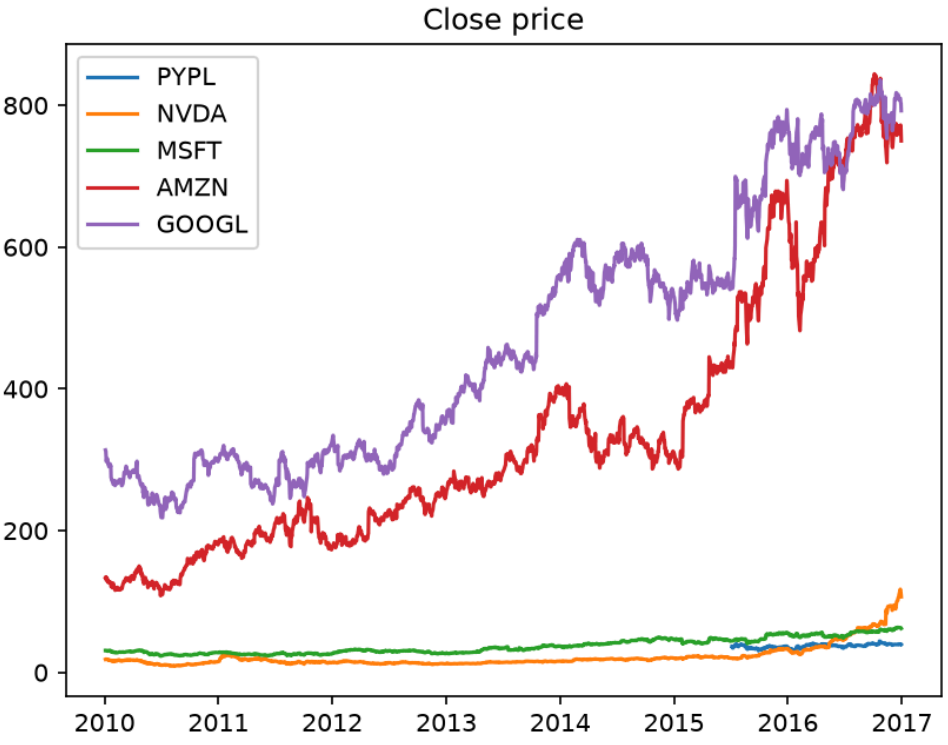
date      0
symbol    0
open      0
close     0
low       0
high      0
volume    0
dtype: int64
501 symbols, 2010-01-04 00:00:00 -> 2016-12-30 00:00:00
['Ticker symbol', 'Security', 'SEC filings', 'GICS Sector', 'GICS Sub Industry', 'Address of Headquarters', 'Date first added', 'CIK']
```

Feature	Mean	Std	Min	25%	Median	75%	Max
open	64.99	75.20	1.66	31.27	48.46	75.12	1,584.44
close	65.01	75.20	1.59	31.29	48.48	75.14	1,578.13
low	64.34	74.46	1.50	30.94	47.97	74.40	1,549.94
high	65.64	75.91	1.81	31.62	48.96	75.85	1,600.93
volume	5.42M	12.49M	0	1.22M	2.48M	5.22M	859.6M

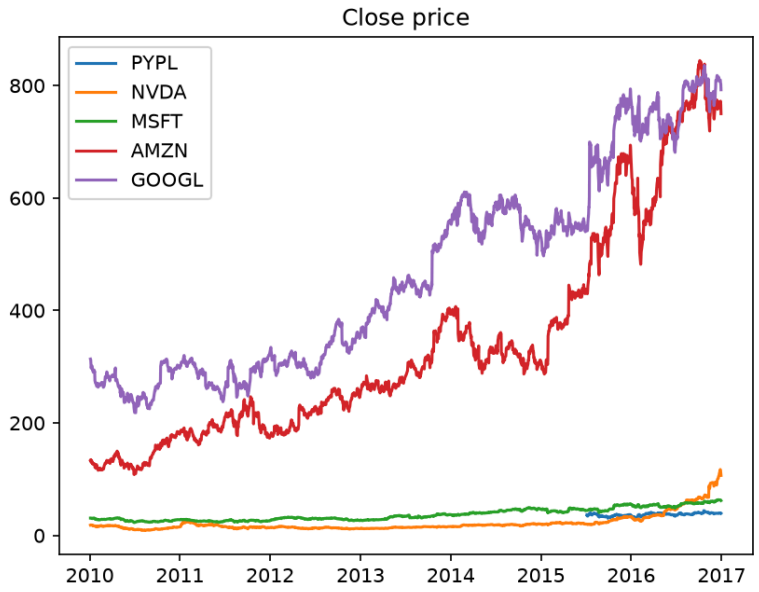
- The four price columns have almost identical statistics, since they describe the same stock on the same day.
- Prices are right-skewed: the mean (about 65) is above the median (about 48), and the maximum (about 1,600) is far above the 75th percentile (about 75). The standard deviation is larger than the mean, so prices vary widely across companies.
- Volume is even more skewed: the mean (5.4M) is more than twice the median (2.5M). The minimum of 0 appears in only 21 of the 851,264 rows, so we kept them, since `log(1 + volume)` handles zeros and their effect is negligible.
- Prices (tens or hundreds) and volume (millions) are on very different scales, so standardization is needed before training a neural network.

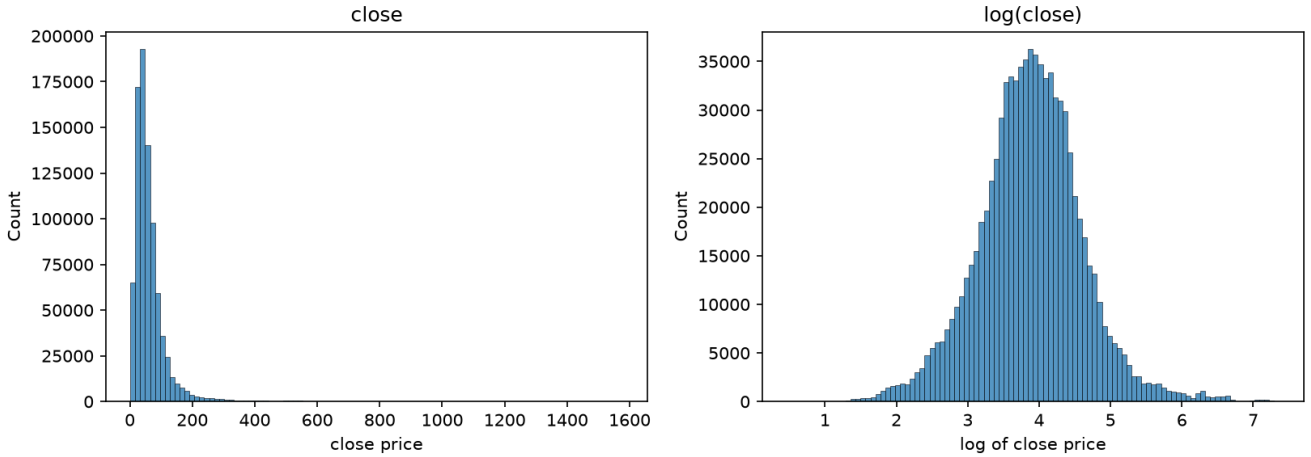


The heatmap shows the correlation between the five numeric features. Open, close, low and high are correlated at 1.00 (rounded), while volume has a weak negative correlation of -0.13 with every price column. The price columns are therefore almost redundant, so a high score on price prediction says little about the model. This is why we compare our results with a naive baseline later.

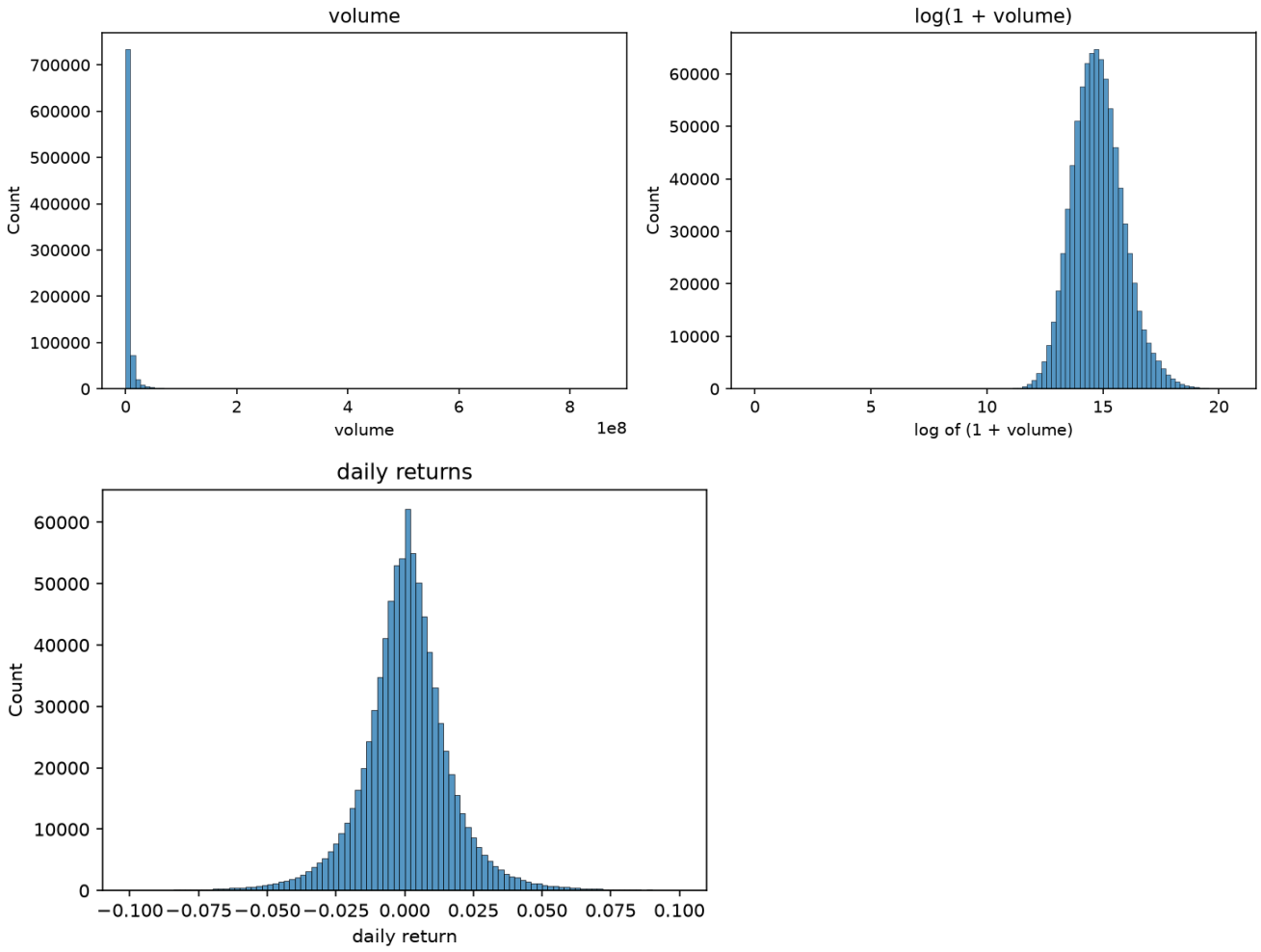


This plot shows the daily closing price of five companies from 2010 to 2016. GOOGL and AMZN trade at several times the price of MSFT, NVDA and PYPL, and PYPL only appears from mid-2015. Price levels and history lengths differ widely across companies.

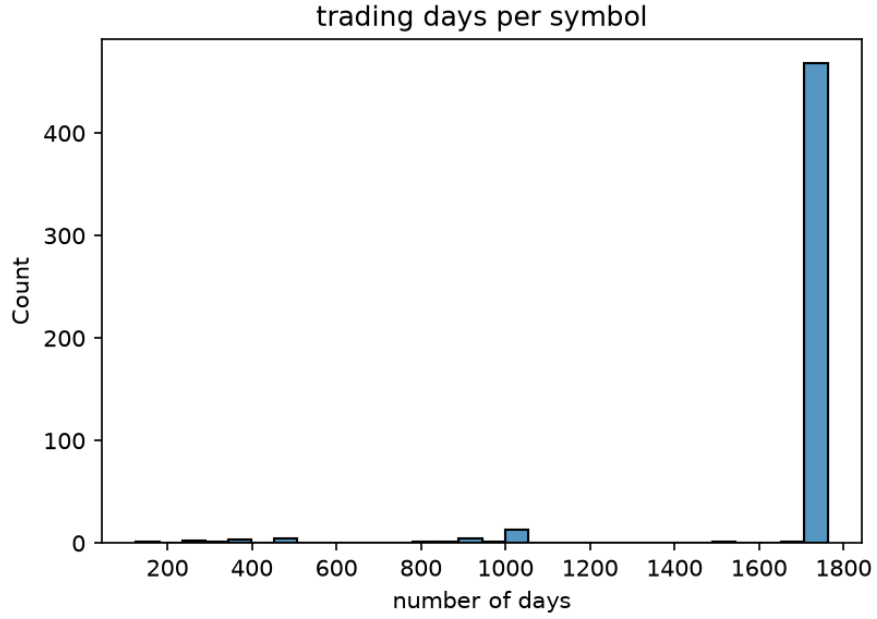




The raw close price is strongly right-skewed: most rows are below about 100, with a long tail up to about 1,600. After a log transform, the distribution is roughly symmetric and bell-shaped.



We compared the raw volume with `log(1 + volume)`, and looked at the daily returns of each company, meaning the change of the close price compared with the previous day. Volume becomes roughly symmetric after the log transform. Daily returns are centred on zero, with most days between -5% and +5%.



Most companies (about 470 of 501) have the full 2010-2016 history of 1,762 trading days. About 30 have shorter histories because they were listed later (the shortest one has 126 days), so the dataset is unbalanced across companies.

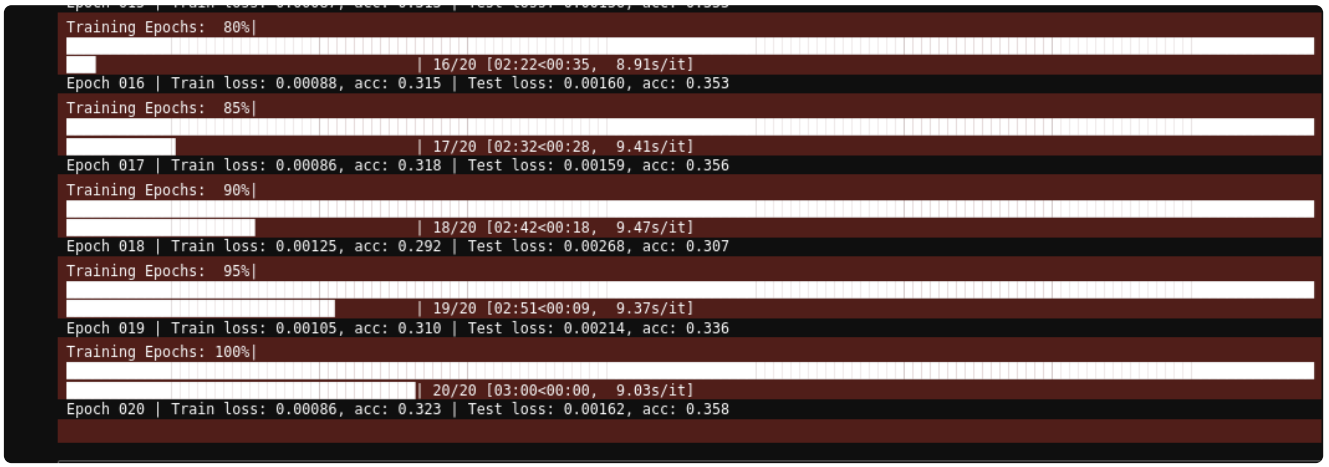
From this analysis we kept four decisions: we standardize the features, we use `log(1 + volume)` instead of raw volume, we split the data by date and not randomly, and we compare the model with a naive baseline.

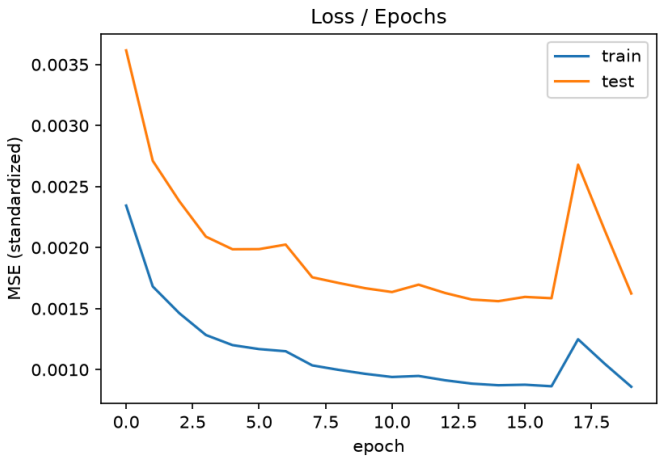
2. Deep Neural Network Architecture for Regression

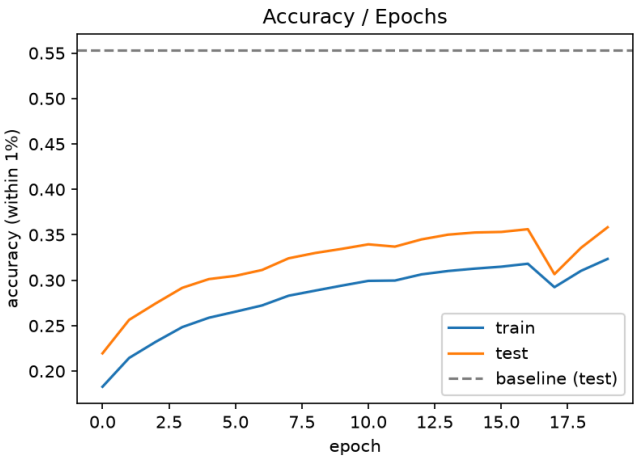
Data preparation. The task is to predict the close price of the next day. We added five features to the four prices: the returns over 1 and 5 days, the relative range of the day, the ratio between the 5-day moving average and the close, and `log(1 + volume)`. The rows without a previous or a next day were removed, which left 848,258 rows. We split the data by date: the first 80% of the rows for training (678,834 rows, up to 2015-08-26) and the last 20% for testing (169,424 rows). Features and target were standardized with a scaler fitted on the training set only, so the test period does not leak into training. Prices drifted upward over time: after scaling, the four price features have a mean of about 0.37 on the test set, against 0 on the training set.

Architecture. We built an MLP in PyTorch with 9 inputs, two hidden layers of 64 and 32 neurons with ReLU, and one output neuron without activation, because the task is a regression. It has 2,753 trainable parameters. We trained it with the MSE loss and SGD (learning rate 0.01, momentum 0.9) on batches of 1,024 rows for 20 epochs, which took about 3 minutes.

Accuracy and baseline. A regression has no natural accuracy, so we define it as the share of predictions within 1% of the true next-day close, measured in dollars. We first tried a 5% tolerance, but the naive baseline "tomorrow's close = today's close" already scored 0.978 on the test set, so it could not separate models. With 1% the baseline scores 0.553.







Model	Test RMSE ($)	Test accuracy (within 1%)
Baseline (tomorrow = today)	2.179	0.553
MLP 9-64-32-1 (SGD, 20 epochs)	2.656	0.358

Interpretation. Both losses decrease and flatten: the train loss goes from 0.00234 to 0.00086 and the test loss from 0.00362 to 0.00162. The test loss stays about twice the train loss, which shows a gap between the training and the test period. Both curves jump up around epoch 18 and recover, so the learning rate seems too large for SGD. The accuracy is still rising at the end (0.323 on train, 0.358 on test), so 20 epochs are not enough to converge. The test accuracy is slightly above the train accuracy even though the test loss is higher, probably because the MSE is dominated by the high-priced stocks while the accuracy is a relative measure. Finally, the first model does not beat the naive baseline: its test RMSE is 2.656 against 2.179, and its accuracy is 0.358 against 0.553. The network has to learn that tomorrow's price is almost today's price, and the baseline gets that for free.

3. Hyper-parameter Search with GridSearch

We used `GridSearchCV` from scikit-learn. Since it only works with scikit-learn models, we wrapped our MLP with the skorch library. To keep the search fast, we ran it on a subsample of the training set (every 7th row, 96,977 rows, which keeps the date order). We used a 3-fold time-series cross-validation, so each fold is validated on dates later than the ones it is trained on. The score is the negative MSE on the standardized target, so the closer to 0 the better.

In the first round we tried 16 combinations (48 fits): the optimizer (SGD with momentum 0.9, or Adam), the learning rate (0.01 or 0.001), the size of the first hidden layer (32 or 64 neurons) and the number of epochs (10 or 20). The best combination was Adam with a learning rate of 0.01, 20 epochs and 64 neurons, with a score of -0.001572. The eight best rows all use a learning rate of 0.01, so 0.001 was too small. The best values were at the edge of our grid, so we ran a second round.

```
[CV] END lr=0.001, max_epochs=20, module__n_hidden1=32, optimizer=<class 'torch.optim.adam.Adam'>; total time=   8.0s
[CV] END lr=0.001, max_epochs=20, module__n_hidden1=32, optimizer=<class 'torch.optim.adam.Adam'>; total time=  10.6s
[CV] END lr=0.001, max_epochs=20, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=   4.0s
[CV] END lr=0.001, max_epochs=20, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=   7.7s
[CV] END lr=0.001, max_epochs=20, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=  13.0s
Best parameters: {'lr': 0.01, 'max_epochs': 20, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
Best score (negative MSE): -0.0015722690150141716
                                          params  mean_test_score  \
11  {'lr': 0.01, 'max_epochs': 20, 'module__n_hidd...        -0.001572
10  {'lr': 0.01, 'max_epochs': 20, 'module__n_hidd...        -0.001621
9   {'lr': 0.01, 'max_epochs': 10, 'module__n_hidd...        -0.002212
8   {'lr': 0.01, 'max_epochs': 10, 'module__n_hidd...        -0.002716
3   {'lr': 0.01, 'max_epochs': 20, 'module__n_hidd...        -0.003114
2   {'lr': 0.01, 'max_epochs': 20, 'module__n_hidd...        -0.003303
1   {'lr': 0.01, 'max_epochs': 10, 'module__n_hidd...        -0.003621
0   {'lr': 0.01, 'max_epochs': 10, 'module__n_hidd...        -0.004500

    std_test_score
11        0.000414
10        0.000500
9         0.000460
8         0.001000
3         0.000988
2         0.000822
1         0.000867
0         0.001370
```

In the second round we kept Adam only and tried 12 combinations (36 fits): learning rates of 0.003, 0.01 and 0.03, 20 or 40 epochs, and 64 or 128 neurons. The best combination was Adam with a learning rate of 0.01, 40 epochs and 64 neurons, with a score of -0.001121 (standard deviation 0.000131). The learning rate of 0.03 was unstable: three of its four combinations had scores around -0.06 with a very large standard deviation, which means the training diverged on some folds. The larger layer of 128 neurons did not help. We also noticed that the same combination (Adam, 0.01, 20 epochs, 64 neurons) scored -0.001572 in the first round and -0.002054 in the second because training is random, so very small differences between scores are not meaningful.

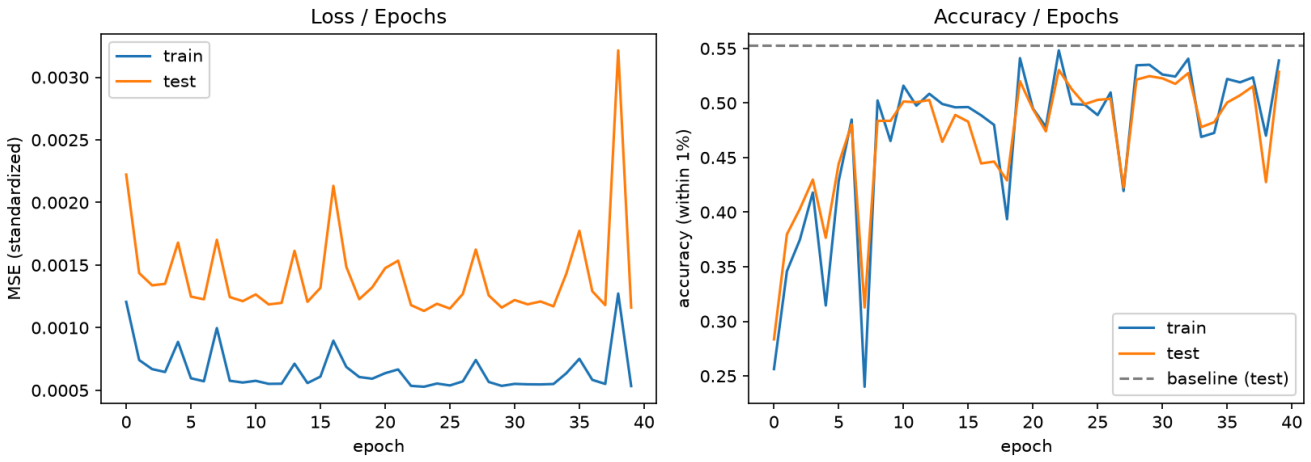
```
[CV] END lr=0.03, max_epochs=20, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=  14.2s
[CV] END lr=0.03, max_epochs=20, module__n_hidden1=128, optimizer=<class 'torch.optim.adam.Adam'>; total time=   5.0s
[CV] END lr=0.03, max_epochs=20, module__n_hidden1=128, optimizer=<class 'torch.optim.adam.Adam'>; total time=   8.0s
[CV] END lr=0.03, max_epochs=20, module__n_hidden1=128, optimizer=<class 'torch.optim.adam.Adam'>; total time=  10.4s
[CV] END lr=0.03, max_epochs=40, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=  11.1s
[CV] END lr=0.03, max_epochs=40, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=  18.6s
[CV] END lr=0.03, max_epochs=40, module__n_hidden1=64, optimizer=<class 'torch.optim.adam.Adam'>; total time=  26.1s
[CV] END lr=0.03, max_epochs=40, module__n_hidden1=128, optimizer=<class 'torch.optim.adam.Adam'>; total time=   9.3s
[CV] END lr=0.03, max_epochs=40, module__n_hidden1=128, optimizer=<class 'torch.optim.adam.Adam'>; total time=  17.7s
[CV] END lr=0.03, max_epochs=40, module__n_hidden1=128, optimizer=<class 'torch.optim.adam.Adam'>; total time=  22.0s
Best parameters: {'lr': 0.01, 'max_epochs': 40, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
Best score (negative MSE): -0.0011206038179807365
                                                                                  params  \
6     {'lr': 0.01, 'max_epochs': 40, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
8     {'lr': 0.03, 'max_epochs': 20, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
4     {'lr': 0.01, 'max_epochs': 20, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
7    {'lr': 0.01, 'max_epochs': 40, 'module__n_hidden1': 128, 'optimizer': <class 'torch.optim.adam.Adam'>}
5    {'lr': 0.01, 'max_epochs': 20, 'module__n_hidden1': 128, 'optimizer': <class 'torch.optim.adam.Adam'>}
1   {'lr': 0.003, 'max_epochs': 20, 'module__n_hidden1': 128, 'optimizer': <class 'torch.optim.adam.Adam'>}
2    {'lr': 0.003, 'max_epochs': 40, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
3   {'lr': 0.003, 'max_epochs': 40, 'module__n_hidden1': 128, 'optimizer': <class 'torch.optim.adam.Adam'>}
0    {'lr': 0.003, 'max_epochs': 20, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
10    {'lr': 0.03, 'max_epochs': 40, 'module__n_hidden1': 64, 'optimizer': <class 'torch.optim.adam.Adam'>}
11   {'lr': 0.03, 'max_epochs': 40, 'module__n_hidden1': 128, 'optimizer': <class 'torch.optim.adam.Adam'>}
9    {'lr': 0.03, 'max_epochs': 20, 'module__n_hidden1': 128, 'optimizer': <class 'torch.optim.adam.Adam'>}

    mean_test_score  std_test_score
6         -0.001121        0.000131
8         -0.001151        0.000395
4         -0.002054        0.000992
7         -0.002147        0.001733
5         -0.002223        0.001104
1         -0.002527        0.000993
2         -0.003537        0.001772
3         -0.003596        0.002748
0         -0.004703        0.003307
10        -0.060418        0.081397
11        -0.065132        0.090645
9         -0.065739        0.091350
```

Answer: the selected hyper-parameters are Adam, a learning rate of 0.01, 40 epochs and hidden layers of 64 and 32 neurons. We did not search the size of the second hidden layer, which stayed at 32.

4. Loss and Accuracy Curves of the Best Model

We retrained the selected model on the full training set for 40 epochs, and recorded the loss and the accuracy on the training and the test set after each epoch.

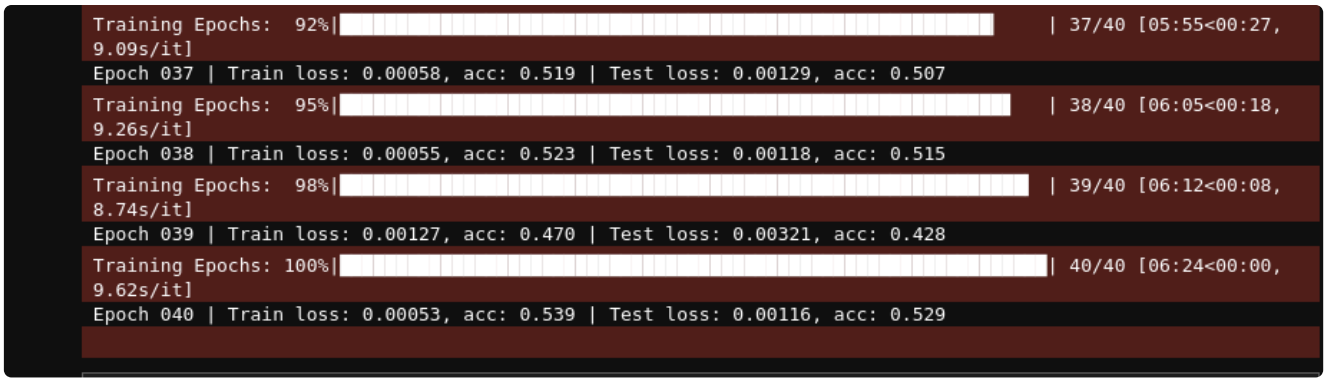


Model	Test RMSE ($)	Test accuracy (within 1%)
Baseline (tomorrow = today)	2.179	0.553
First model (SGD, 20 epochs)	2.656	0.358
Best model (Adam, 40 epochs)	2.246	0.529

Interpretation. The train loss falls from 0.00121 after the first epoch to 0.00053 at the end, and the test loss from 0.00222 to 0.00116. The test loss stays about twice the train loss during the whole training, so the gap is stable and does not grow, which means the model is not overfitting. The accuracy climbs from 0.256 (train) and 0.283 (test) to 0.539 and 0.529. Both curves show sharp spikes, for example at epochs 5, 8, 17 and 39, where the loss jumps up and the accuracy drops before recovering. We think the learning rate of 0.01 is too large for Adam once the model is close to a minimum, so the final value depends on the epoch where the training stops: the mean test accuracy over the last 5 epochs is 0.496, against 0.529 at the last epoch. Compared with the first model, the test RMSE improves from 2.656 to 2.246 dollars and the accuracy from 0.358 to 0.529. The model gets close to the naive baseline but does not beat it.

5. Regularization Techniques

We applied three regularization techniques to the best model, with the same seed, architecture, optimizer and 40 epochs: L2 weight decay (1e-4), dropout (p = 0.2) and batch normalization. We also combined dropout with L2. We compare them with the model without regularization, which is itself better than the first model (test accuracy 0.358).



Model	Train acc	Test acc	Test acc (mean of last 5 epochs)	Test RMSE ($)
No regularization (best model)	0.539	0.529	0.496	2.246
L2 (weight decay 1e-4)	0.510	0.500	0.477	2.295
Dropout (p=0.2)	0.089	0.135	0.108	8.401
Batch normalization	0.314	0.354	0.364	2.494
Dropout + L2	0.147	0.180	0.132	8.513

Interpretation. None of the techniques improved the model. L2 came closest (test accuracy 0.500 and RMSE 2.295 dollars against 0.529 and 2.246 without regularization), batch normalization was worse (0.354), and dropout was much worse: even the training accuracy drops to 0.089. Regularization is meant to reduce overfitting, and the best model was not overfitting, since its train and test accuracy are almost equal, so there was nothing for it to fix. Dropout most likely hurts because its noise makes it hard for such a small network to predict the price within 1%. We did not test other dropout rates.

Part 2 - Multi-class Classification

We used the Kaggle dataset "Machine Predictive Maintenance Classification", stored in the folder ../machine-predictive. The goal is to predict the type of failure of a machine from sensor measurements.

1. Pre-processing

The file `predictive_maintenance.csv` has 10,000 rows and 10 columns: an identifier (`UDI`, `Product ID`), the machine quality `Type` (L, M or H), five sensor measurements (air temperature, process temperature, rotational speed, torque and tool wear), the binary flag `Target` and the label `Failure Type`. There are no missing values and no duplicated rows.

We compared `Target` with `Failure Type`. Nine rows have `Target = 1` but the label "No Failure", so the two columns contradict each other and we removed these rows, which left 9,991 rows. All 18 "Random Failures" have `Target = 0`. These failures are random events, so they may not be predictable from the sensors, but we kept them because they are one of

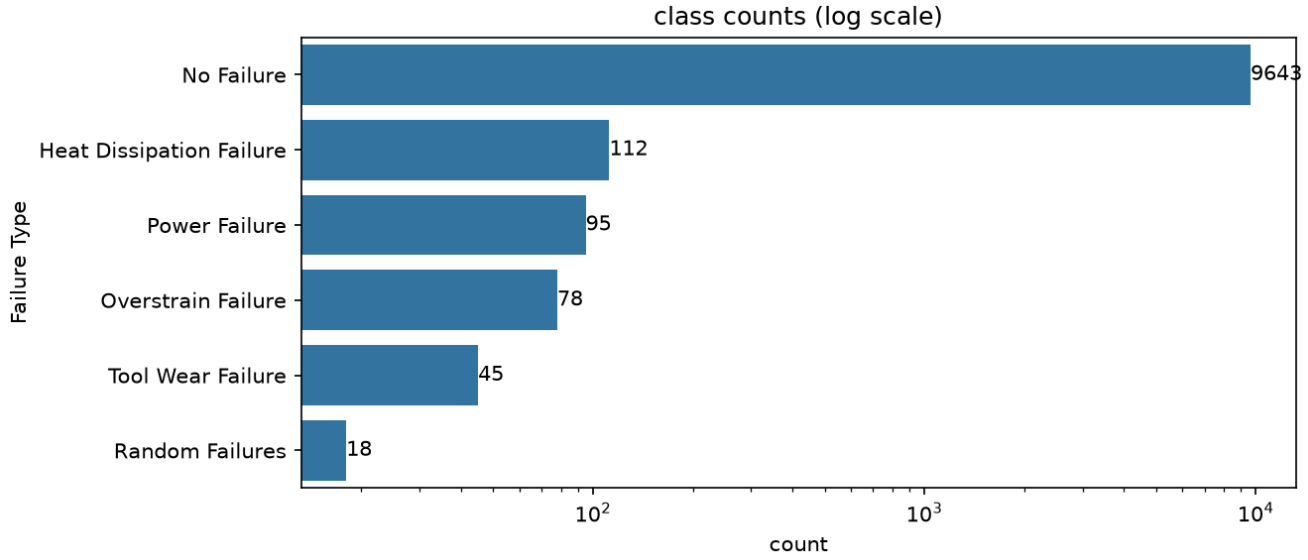
the classes of the problem. We dropped `UDI` and `Product ID`, which are only identifiers, and `Target`, which is a binary version of the label and would leak the answer to the model. `Type` was encoded as 0, 1 and 2 for L, M and H.

We encoded the six labels and split the data into 80% for training and 20% for testing, with the same class proportions in both sets (stratified split).

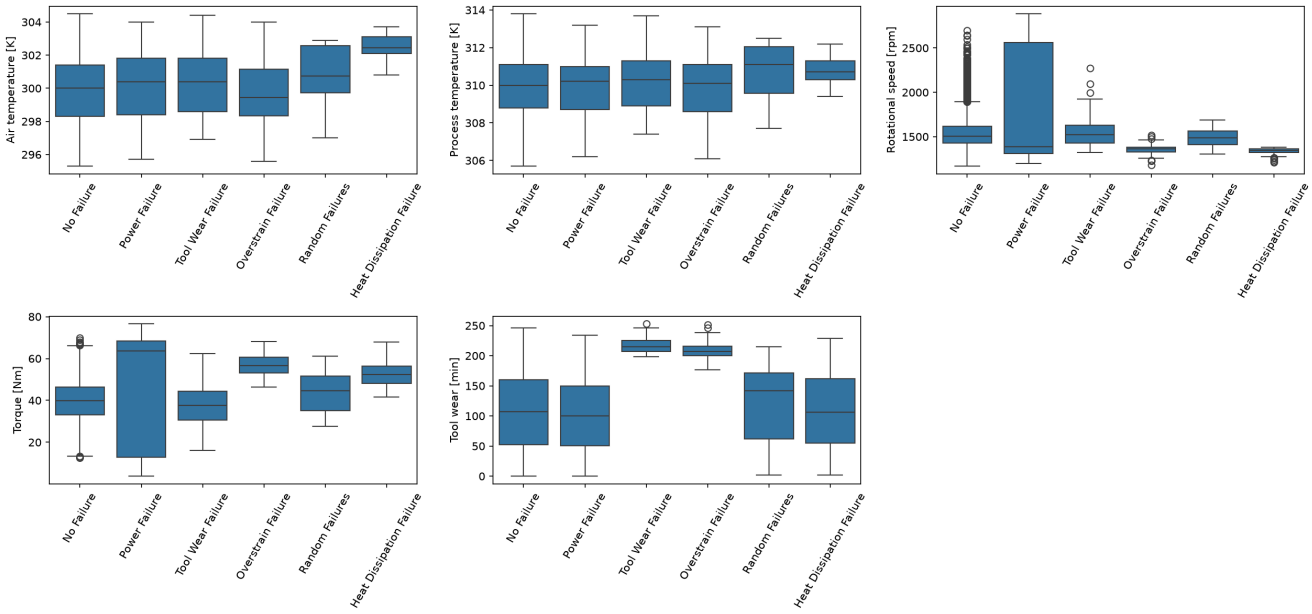
Class	Train	Test
Heat Dissipation Failure	90	22
No Failure	7,714	1,929
Overstrain Failure	62	16
Power Failure	76	19
Random Failures	14	4
Tool Wear Failure	36	9

The test set contains only 4 Random Failures and 9 Tool Wear Failures, so the metrics of these two classes will be very noisy. We then standardized the five numeric columns with a scaler fitted on the training set only, so the test set does not influence it. After scaling, the training means are 0, and the test means are between -0.04 and 0.03, which is expected since the test set uses the training mean and standard deviation. `Type` was left unscaled.

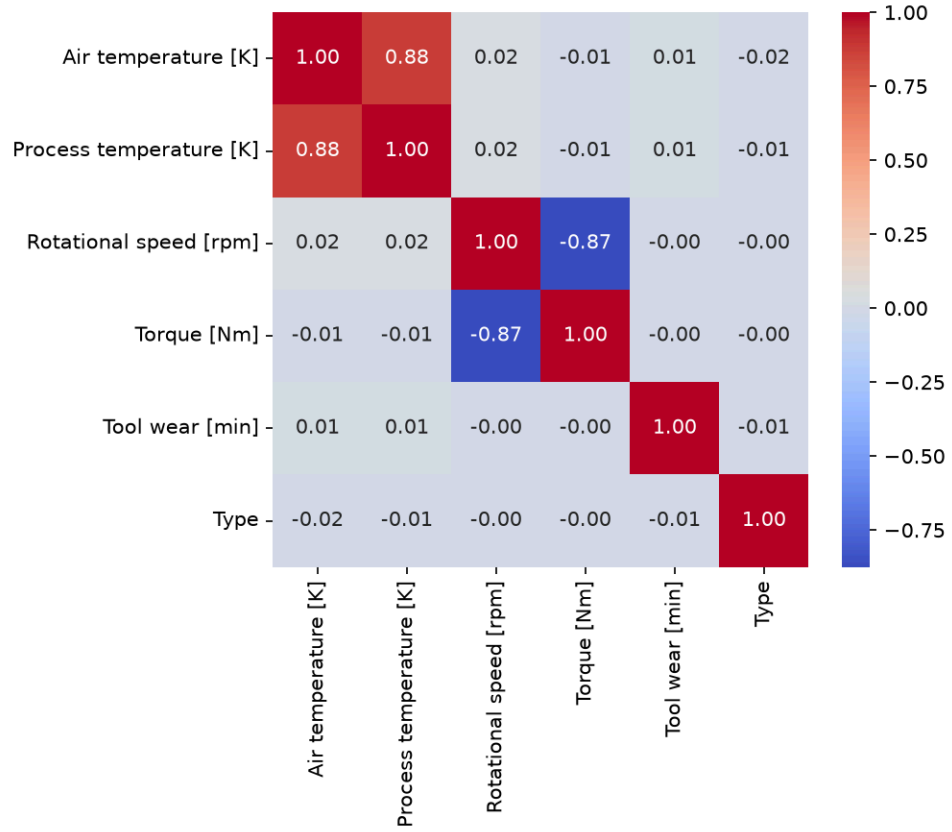
2. Exploratory Data Analysis



The dataset is extremely imbalanced: 9,643 of the 9,991 rows (96.5%) are "No Failure", and the failure classes have 112, 95, 78, 45 and 18 rows. A log scale is needed to see the small classes. For this reason accuracy alone is not a good metric, because a model predicting "No Failure" every time would already be right on 96.5% of the rows.



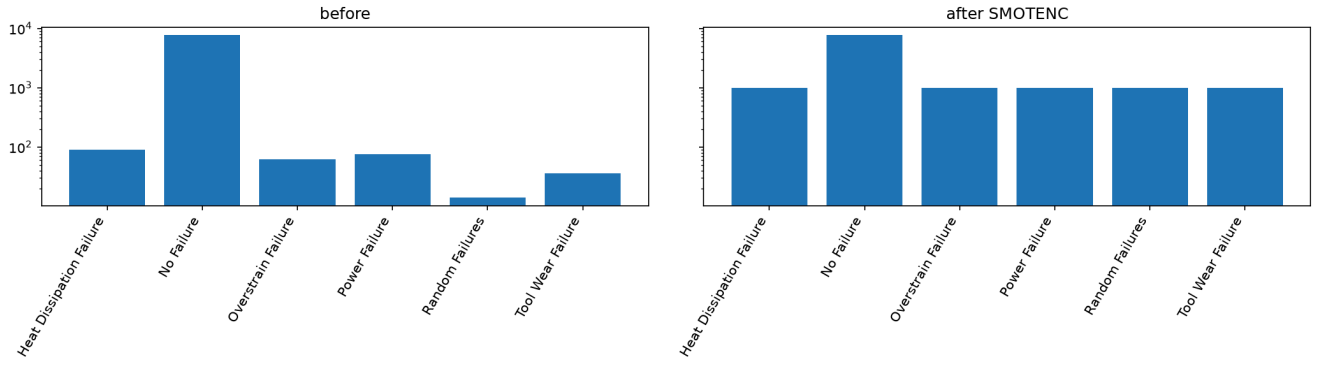
The boxplots show that the classes are partly separable. Heat Dissipation failures have a higher air temperature and a low, narrow rotational speed. Overstrain failures have a high torque, a low speed and a high tool wear. Tool Wear failures have a high and narrow tool wear. Power failures cover a very wide range of speed and torque. Random Failures largely overlap with "No Failure", so we expect them to be hard to predict.



Air temperature and process temperature are strongly correlated (0.88), and rotational speed and torque are strongly anti-correlated (-0.87). The other pairs are close to 0, and `Type` is not correlated with any feature.

3. Data Augmentation to Balance the Dataset

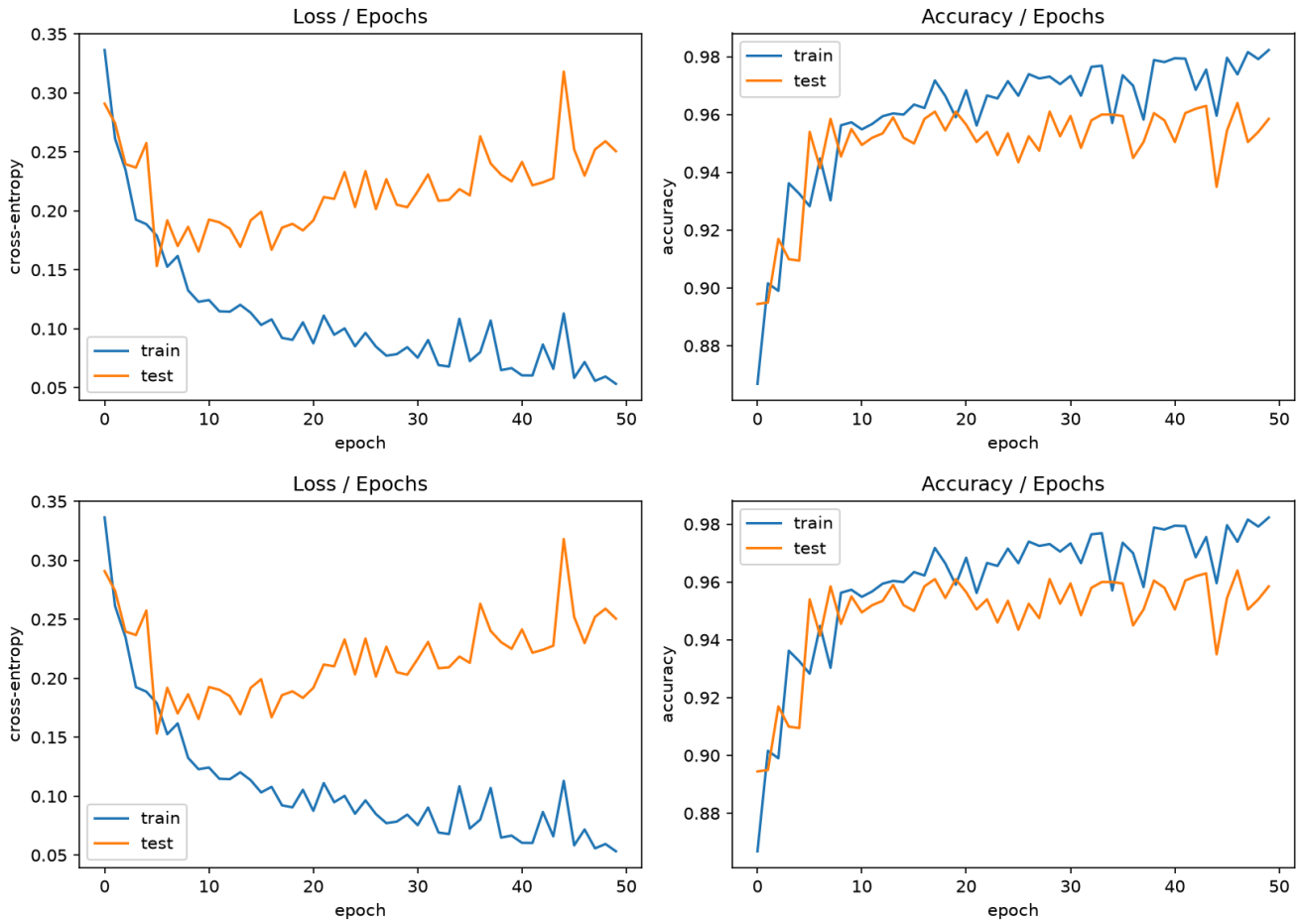
We balanced the training set with SMOTENC, which creates synthetic samples between neighbouring real samples and handles the categorical column `Type`, so the synthetic rows keep a valid value of 0, 1 or 2. We raised each of the five failure classes to 1,000 samples and left "No Failure" unchanged at 7,714, so the training set has 12,714 rows. We did not balance up to 7,714 per class, because it would create thousands of synthetic rows from as few as 14 real ones. The test set was not touched, so it keeps the real class distribution.



The synthetic rows of the smallest classes are interpolations of only 14 to 90 real samples, so these classes remain unreliable.

4. Deep Neural Network Architecture for Multi-class Classification

We built an MLP in PyTorch with 6 inputs, two hidden layers of 64 and 32 neurons with ReLU, and 6 outputs, one per class. The model returns raw scores and we use the cross-entropy loss, which applies the softmax itself, and the predicted class is the one with the highest score. We trained it with SGD (learning rate 0.01, momentum 0.9) on batches of 32 rows for 50 epochs, on the balanced training set.



Interpretation. The train loss keeps decreasing, from about 0.37 to 0.0585, while the test loss reaches its lowest values (about 0.15 to 0.17) within the first 10 epochs and then oscillates between 0.17 and 0.27, ending at 0.2291. The gap keeps growing, which is a sign of overfitting. At epoch 50 the train accuracy is 0.980 (on the balanced training set) and the test accuracy is 0.953. This test accuracy is even below 0.965, the score of a model that always predicts "No Failure" (1,929 of the 1,999 test rows), so we also follow the macro F1, which gives the same weight to each class. It is only 0.535 on the test set at epoch 50, which shows that the rare classes are not learned well.

5. Hyper-parameter Search with GridSearch

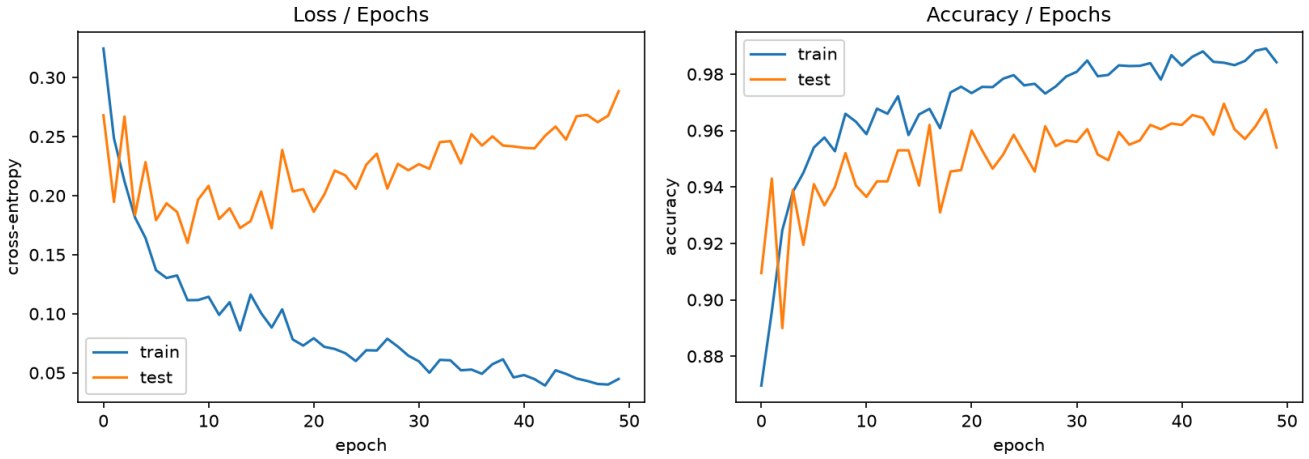
We used `GridSearchCV` from scikit-learn with a small wrapper class that lets scikit-learn train and predict with our PyTorch MLP. The search tried 32 combinations (96 fits with a 3-fold cross-validation): the sizes of the two hidden layers (64 or 128, and 32 or 64 neurons), the learning rate (0.01 or 0.001), the optimizer (SGD or Adam) and the number of epochs (30 or 50). We scored with the macro F1, because accuracy hides the rare classes.

The best combination was Adam, a learning rate of 0.001, 50 epochs and hidden layers of 128 and 64 neurons, with a cross-validated macro F1 of 0.6311. All the best values are at the edge of our grid (the largest layers and number of epochs, the smallest learning rate), so a better setting may exist beyond it. We also note that the search was run on the already balanced training set, so synthetic rows made from the same real sample can appear in both a training fold and a validation fold. The score of 0.6311 is therefore optimistic, and higher than the macro F1 we obtain later on the test set.

Answer: the selected hyper-parameters are Adam, a learning rate of 0.001, 50 epochs and hidden layers of 128 and 64 neurons.

6. Loss and Accuracy Curves of the Best Model

We trained the model with the selected hyper-parameters on the balanced training set for 50 epochs, and recorded the loss and the accuracy on the training and the test set after each epoch.



7. Metrics on the Training and Test Sets

We computed the accuracy, the sensitivity (recall), the precision and the F1 score on the training set and on the test set. For the training set we give the balanced set that the model learned from, and also the original unbalanced one, which is the one comparable with the test set.

Set	Accuracy	Sensitivity (macro)	Precision (macro)	F1 (macro)	F1 (weighted)
Train (balanced)	0.9842	0.9910	0.9717	0.9810	0.9843
Train (original)	0.9782	0.9857	0.6777	0.7642	0.9827
Test	0.9540	0.6235	0.4926	0.5461	0.9640

Metrics per class on the test set:

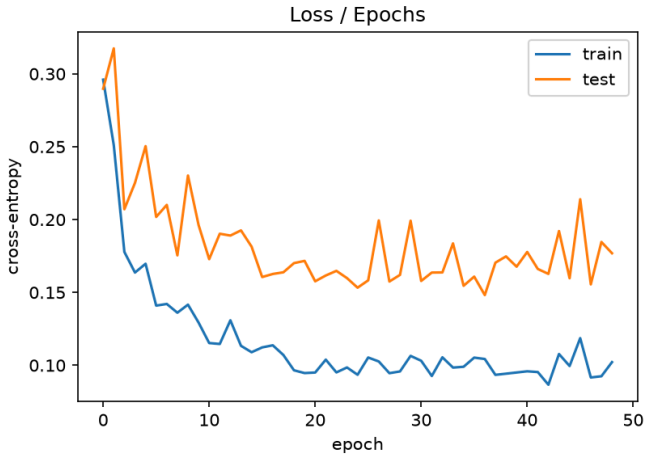
Class	Precision	Sensitivity	F1	Support
Heat Dissipation Failure	0.667	1.000	0.800	22
No Failure	0.992	0.961	0.976	1,929
Overstrain Failure	0.682	0.938	0.789	16
Power Failure	0.615	0.842	0.711	19
Random Failures	0.000	0.000	0.000	4

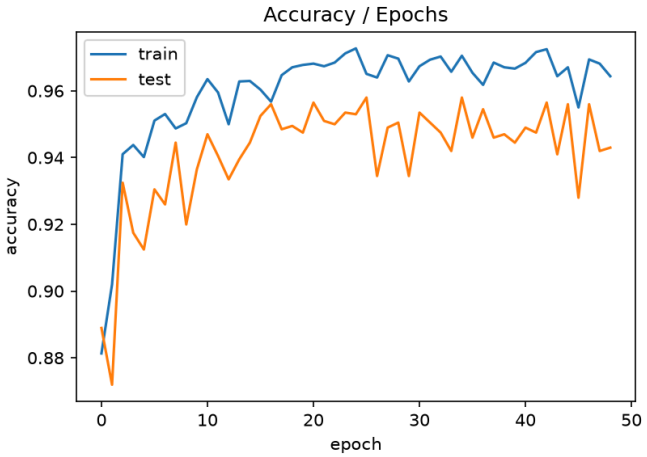
Class	Precision	Sensitivity	F1	Support
Tool Wear Failure	0.000	0.000	0.000	9

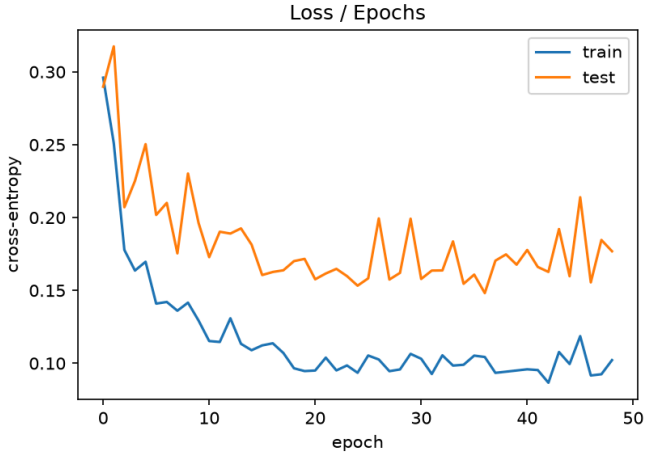
Interpretation. The weighted scores are high (0.964 weighted F1 on the test set) only because "No Failure" dominates the test set. The macro scores tell the real story: the macro F1 drops from 0.764 on the original training set to 0.546 on the test set, and the macro sensitivity from 0.986 to 0.624. On the test set, the model finds most Heat Dissipation (sensitivity 1.000), Overstrain (0.938) and Power failures (0.842), but it never detects the Random Failures or the Tool Wear failures. On the training set the sensitivity of these two classes is 1.000 (and the precision only 0.318 and 0.279), so the model has most likely memorized the few real examples and their synthetic copies without learning a rule that generalizes. For Random Failures this is consistent with the overlap with "No Failure" seen in the analysis. With only 4 and 9 test rows for these classes, these values are also very noisy.

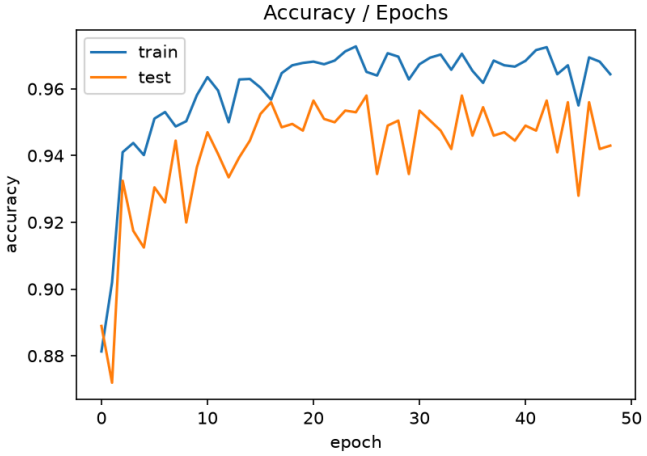
8. Regularization

We applied several regularization techniques together to the best architecture (128 and 64 neurons): dropout with a rate of 0.1, L2 weight decay of 5e-4 and early stopping with a patience of 12 epochs, which keeps the weights of the epoch with the lowest loss. We trained it with Adam and a learning rate of 0.003 for at most 60 epochs. The early stopping monitors the loss on the test set, so the stopping epoch was chosen with the test data, which is a limitation of this comparison.


Loss / Epochs
train
test
cross-entropy
epoch
0.30
0.25
0.20
0.15
0.10
0
10
20
30
40
50


Accuracy / Epochs
train
test
accuracy
epoch
0.96
0.94
0.92
0.90
0.88
0
10
20
30
40
50


Loss / Epochs
train
test
cross-entropy
epoch
0.30
0.25
0.20
0.15
0.10
0
10
20
30
40
50


Accuracy / Epochs
train
test
accuracy
epoch
0.96
0.94
0.92
0.90
0.88
0
10
20
30
40
50